

3Sum problem

12.8

nums = [-1, 0, 1, 2, -1, 4]

$$\text{Sum} = i + j + k == 0 \quad \begin{cases} i \neq j \\ i \neq k \\ j \neq k. \end{cases}$$

Brute force Approach

```
for (int i = 0; i < nums.length; i++) {  
    for (int j = i + 1; j < nums.length; j++) {  
        for (int k = j + 1; k < nums.length; k++) {  
  
            if (nums[i] + nums[j] + nums[k] == 0) {  
  
                res.add(Arrays.asList(nums[i],  
                                       nums[j],  
                                       nums[k]));  
  
            }  
  
        }  
    }  
}  
  
return res;
```

Time Complexity = $O(n^3)$.

As need to optimized this
So, we use two pointer approach!

Optimum Approach:

nums = [-1, 0, 1, 2, -1, -4]

$\uparrow$ $\uparrow$ $\uparrow$
i *l* *r*
 $\downarrow$ $i+1$ $\downarrow$ $n-1$

nums ko ascending order me sort krsko.

nums = [-4, -1, 0, 1, 2]

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
i ~~*l*~~ ~~*l*~~ ~~*l*~~ ~~*l*~~ ~~*l*~~

$$\text{sum} = -4 + (-1) + 2 \neq 0 \quad \times$$

$$\text{sum} = -4 + (-1) + 1 \neq 0 \quad \times$$

nums = [-4, -1, -1, 0, 1, 2]

$\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$ $\uparrow$
i ~~*l*~~ ~~*l*~~ *r* ~~*l*~~

$$\text{sum} = (-1) + (-1) + 2 = 0 \quad \checkmark$$

$$\text{sum} = (-1) + 0 + 1 = 0 \quad \checkmark$$

output = [-1, -1, 2] ~~[-1, 0, 1]~~

nums = [-4, -1, -1, 0, 1, 2]

$\uparrow$ $\uparrow$ $\uparrow$
i *l* *r*

$$\text{sum} = -1 + 0 + 2 \neq 0 \quad \times$$

Age possible triplets nhi hai.

// Code

class Sol {

public List<List<Integer>> threeSum(int[] nums) {

List<List<Integer>> res = new ArrayList<>();

int n = nums.length;

Arrays.sort(nums);

for (int i = 0; i < n; i++) {

if (i > 0 && nums[i] == nums[i-1]) continue;

int l = i+1;

int r = n-1;

while (l < r) {

int sum = nums[i] + nums[l] + nums[r];

if (sum == 0) {

res.add(Arrays.asList(nums[i], nums[l],
 nums[r]));

while (l < r && nums[l] == nums[l+1]) l++;
 // Isme agar left side duplicate element ko remove
 kar dega.

while (l < r && nums[r] == nums[r-1]) r--;
 // Isme right side se duplicate element ko remove
 kar dega.

l++;

r--;

}

else if (sum > 0) {
 r--;

} else {

l++;

}

}
 return res;

Time Complexity
is = $O(n) \times O(n^2)$
inner outer
 $O(n^3)$